



HOOT

Overview

HOOT is a testing framework written with Owl whose key features are:

- to register and run tests and test suites;
- to display an intuitive interface to view and filter test results;
- to provide ways to interact with the DOM to simulate user actions;
- to provide low-level helpers allowing to mock various global objects.

As such, it has been integrated as a *lib/* in the Odoo codebase and exports 2 main modules:

- `@odoo/hoot-dom` : (can be used in tours) helpers to:
 - **interact** with the DOM, such as `click()` and `press()` ;
 - **query** elements from the DOM, such as `queryAll()` and `waitFor()` ;
- `@odoo/hoot` : (only to be used in unit tests) all the test framework features:
 - `test`, `describe` and `expect`
 - test hooks like `after` and `afterEach`
 - fixture handling with `getFixture`
 - date and time handling like `mockDate` or `advanceTime`
 - mocking network responses through `mockFetch()` or `mockWebSocket()`
 - every helper exported by `@odoo/hoot-dom`

Note

This section of the documentation is not meant to list *all* helpers available in Hoot (the full list can be found in the [@odoo/hoot](#) module itself). The goal here is to showcase the most used helpers and to justify some of the decisions that have led to the current shape of the testing framework.

Running tests

In Odoo, frontend unit tests can be run by going to the `/web/tests` URL. Most of the setup for calling the test runner is already in place:

function.

When going to the test page, tests will be run sequentially and the results will be displayed in the console and in the GUI (if not running in `headless` mode).

Runner options

The runner can be configured either:

- through the interface (with the configuration dropdown and the search bar);
- or through the URL query parameters (e.g. `?headless` to run in headless mode).

Here is the list of available options for the runner:

- **bail**
Amount of failed tests after which the test runner will be stopped. A falsy value (including 0) means that the runner should never be aborted. (default: 0)
- **debugTest**
Same as the `FILTER_SCHEMA.test` filter, while also putting the test runner in "debug" mode. See `TestRunner.debug` for more info. (default: false)
- **fps**
Sets the value of frames per seconds (this will be transformed to milliseconds and used in `advanceFrame`)
- **filter**
Search string that will filter matching tests/suites, based on their full name (including their parent suite(s)) and their tags. (default: "")
- **frameRate**
Estimated amount of frames rendered per second, used when mocking animation frames. (default: 60 fps)
- **fun**
Lightens the mood. (default: false)
- **headless**
Whether to render the test runner user interface. (default: false)
- **id**

Log level used by the test runner. The higher the level, the more logs will be displayed:

- 0 : only runner logs are displayed (default)
- 1 : all suite results are also logged
- 2 : all test results are also logged
- 3 : debug information for each test is also logged

- **manual**

Whether the test runner must be manually started after page load (defaults to starting automatically). (default: `false`)

- **notrycatch**

Removes the safety of `try .. catch` statements around each test's run function to let errors bubble to the browser. (default: `false`)

- **order**

Determines the order of test execution:

- "fifo" : tests will be run sequentially as declared in the file system;
- "lifo" : tests will be run sequentially in the reverse order;
- "random" : shuffles tests and suites within their parent suite.

- **preset**

Environment in which the test runner is running. This parameter is used to determine the default value of other features, namely:

- the user agent;
- touch support;
- expected size of the viewport.

- **showdetail**

Determines how the failed tests must be unfolded in the UI. (default: `"first-fail"`)

- **tag**

Tag names of tests and suites to run exclusively (case insensitive). (default: empty)

- **timeout**

Duration (in milliseconds) at the end of which a test will automatically fail. (default: 5 seconds)

Note

matching tests from the list of tests to run).

Writing tests

Test

Writing a test can be very straightforward, as it is just a matter of calling the `test` function with a name and a function that will contain the test logic.

Here is a simple example:

```
import { expect, test } from "@odoo/hoot";

test("My first test", () => {
  expect(2 + 2).toBe(4);
});
```

Describe

Most of the time, tests are not that simple. They often require some setup and teardown, and sometimes they need to be grouped together in a suite. This is where the `describe` function comes into play.

Here is how you would declare a suite and a test within it:

```
import { describe, expect, test } from "@odoo/hoot";

describe("My first suite", () => {
  test("My first test", () => {
    expect(2 + 2).toBe(4);
  });
});
```

Important

In Odoo, all test files are run in an isolated environment and are wrapped within a global `describe` block (with the name of the suite being the *path* of the test file).

Expect

The `expect` function is the main assertion function of the framework. It is used to assert that a value or an object is what it is expected to be or in the state it is supposed to be. To do so, it provides a few modifiers and a wide range of matchers.

Modifiers

An `expect` modifier is a getter that returns another set of *altered* matchers that will behave in a specific way.

- **not**

Inverts the result of the following matcher: it will succeed if the matcher fails.

```
expect(true).not.toBe(false);
```

- **resolves**

Waits for the value (Promise) to be *"resolved"* before running the following matcher with the resolved value.

```
await expect(Promise.resolve(42)).resolves.toBe(42);
```

- **rejects**

Waits for the value (Promise) to be *"rejected"* before running the following matcher with the rejected reason.

```
await expect(Promise.reject("error")).rejects.toBe("error");
```

Note

The `resolves` and `rejects` modifiers are only available when the value is a promise, and will return a promise that will resolve once the assertion is done.

Regular matchers

Note that the last argument parameter of all matchers is an optional dictionary with additional options, in which a custom assertion message can be given for added context/specificity.

The first list of matchers are primitive or object based and are the most common ones:

toBe(expected[, options])

Expects the received value to be *strictly equal* to the expected value.

- Parameters
 - expected : any
 - options : { message?: string }

- Examples

```
expect("foo").toBe("foo");  
expect({ foo: 1 }).not.toBe({ foo: 1 });
```

toBeCloseTo(expected[, options])

Expects the received value to be *close to* the expected value up to a given amount of digits (default is 2).

- Parameters
 - expected : any
 - options : { message?: string, digits?: number }

- Examples

```
expect(0.2 + 0.1).toBeCloseTo(0.3);  
expect(3.51).toBeCloseTo(3.5, { digits: 1 });
```

toBeEmpty([options])

Expects the received value to be empty:

- iterable : no items
- object : no keys

- `options: { message?: string }`

- Examples

```
expect({}).toBeEmpty();  
expect(["a", "b"]).not.toBeEmpty();  
expect(queryOne("input")).toBeEmpty();
```

`toBeGreaterThan(min[, options])`

Expects the received value to be *strictly greater* than `min`.

- Parameters

- `min: number`
- `options: { message?: string }`

- Examples

```
expect(5).toBeGreaterThan(-1);  
expect(4 + 2).toBeGreaterThan(5);
```

`toBeInstanceOf(cls[, options])`

Expects the received value to be an instance of the given `cls`.

- Parameters

- `cls: Function`
- `options: { message?: string }`

- Examples

```
expect({ foo: 1 }).not.toBeInstanceOf(Object);  
expect(document.createElement("div")).toBeInstanceOf(HTMLElement);
```

`toBeLessThan(max[, options])`

- `max`: number
- `options`: { `message?`: string }

- Examples

```
expect(5).toBeLessThan(10);  
expect(8 - 6).toBeLessThan(3);
```

`toBeOfType(type[, options])`

Expects the received value to be of the given `type`.

- Parameters
 - `type`: string
 - `options`: { `message?`: string }

- Examples

```
expect("foo").toBeOfType("string");  
expect({ foo: 1 }).toBeOfType("object");
```

`toBeWithin(min, max[, options])`

Expects the received value to be *between* `min` and `max` (both inclusive).

- Parameters
 - `min`: number
 - `max`: number
 - `options`: { `message?`: string }

- Examples

```
expect(3).toBeWithin(3, 9);  
expect(-8.5).toBeWithin(-20, 0);  
expect(100).toBeWithin(50, 100);
```


- Parameters
 - `expected`: any
 - `options`: { `message?`: string }

- Examples

```
expect(["foo"]).toEqual(["foo"]);
expect({ foo: 1 }).toEqual({ foo: 1 });
```

`toHaveLength(length[, options])`

Expects the received value to have a length of the given `length`. Received value can be any `Iterable` or `Object`.

- Parameters
 - `length`: number
 - `options`: { `message?`: string }

- Examples

```
expect("foo").toHaveLength(3);
expect([1, 2, 3]).toHaveLength(3);
expect({ foo: 1, bar: 2 }).toHaveLength(2);
expect(new Set([1, 2])).toHaveLength(2);
```

`toContain(item[, options])`

Expects the received value to include an `item` of a given shape.

Received value can be an iterable or an object (in case it is an object, the `item` should be a key or a tuple representing an entry in that object).

Note that it is NOT a strict comparison: the item will be matched for deep equality against each item of the iterable.

- Parameters
 - `item`: any

```
expect([1, 2, 3]).toInclude(2);
expect({ foo: 1, bar: 2 }).toInclude("foo");
expect({ foo: 1, bar: 2 }).toInclude(["foo", 1]);
expect(new Set([{ foo: 1 }, { bar: 2 }])).toInclude({ bar: 2 });
```

toMatch(*matcher*[, *options*])

Expects the received value to match the given `matcher`.

- Parameters
 - `matcher`: `string` | `number` | `RegExp`
 - `options`: { `message?: string` }
- Examples

```
expect(new Error("foo")).toMatch("foo");
expect("a foo value").toMatch(/fo.*ue/);
```

toThrow(*matcher*[, *options*])

Expects the received `Function` to throw an error after being called.

- Parameters
 - `matcher`: `string` | `number` | `RegExp`
 - `options`: { `message?: string` }
- Examples

```
expect(() => { throw new Error("Woops!") }).toThrow(/woops/i);
await expect(Promise.reject("foo")).rejects.toThrow("foo");
```

DOM matchers

This next list of matchers are node-based and are used to assert the state of a node or a list of nodes. They generally take a **custom selector** as the argument of the `expect` function (although a `Node` or an iterable of `Node` is also accepted).

- Parameters

- `options: { message?: string, indeterminate?: boolean }`

- Examples

```
expect("input[type=checkbox]").toBeChecked();
```

toBeDisplayed([options])

Expects the received `Target` to be *"displayed"*, meaning that:

- it has a bounding box;
- it is contained in the root document.

- Parameters

- `options: { message?: string }`

- Examples

```
expect(document.body).toBeDisplayed();  
expect(document.createElement("div")).not.toBeDisplayed();
```

toBeEnabled([options])

Expects the received `Target` to be *"enabled"*, meaning that it matches the `:enabled` pseudo-selector.

- Parameters

- `options: { message?: string }`

- Examples

```
expect("button").toBeEnabled();  
expect("input[type=radio]").not.toBeEnabled();
```

- Parameters

- `options: { message?: string }`

toBeVisible([options])

Expects the received `Target` to be “*visible*”, meaning that:

- it has a bounding box;
- it is contained in the root document;
- it is not hidden by CSS properties.

- Parameters

- `options: { message?: string }`

- Examples

```
expect(document.body).toBeVisible();  
expect("[style='opacity: 0']").not.toBeVisible();
```

toHaveAttribute(attribute, value[, options])

Expects the received `Target` to have the given attribute set, and for that attribute value to match the given `value` if any.

- Parameters

- `attribute: string`
 - `value: string | number | RegExp`
 - `options: { message?: string }`

- Examples

```
expect("a").toHaveAttribute("href");  
expect("script").toHaveAttribute("src", "./index.js");
```

toHaveClass(className[, options])

Expects the received `Target` to have the given class name(s).

- Examples

```
expect("button").toHaveClass("btn btn-primary");
expect("body").toHaveClass(["o_webclient", "o_dark"]);
```

toHaveCount(*amount*[, *options*])

Expects the received Target to contain exactly *amount* element(s). Note that the *amount* parameter can be omitted, in which case the function will expect *at least* one element.

- Parameters

- *amount*: number
- *options*: { message?: string }

- Examples

```
expect(".o_webclient").toHaveCount(1);
expect(".o_form_view .o_field_widget").toHaveCount();
expect("ul > li").toHaveCount(4);
```

toHaveInnerHTML(*expected*[, *options*])

Expects the *innerHTML* of the received Target to match the *expected* value (upon formatting).

- Parameters

- *expected*: string | RegExp
- *options*:
{ message?: string, type?: "html" | "xml", tabSize?: number,
keepInlineTextNodes?: boolean }

- Examples

```
expect(".my_element").toHaveInnerHTML(`
  Some <strong>text</strong>
```

Expects the `outerHTML` of the received `Target` to match the `expected` value (upon formatting).

- Parameters
 - `expected`: `string` | `RegExp`
 - `options`:
 - `{ message?: string, type?: "html" | "xml", tabSize?: number, keepInlineTextNodes?: boolean }`
- Examples

```
expect(".my_element").toHaveOuterHTML(`
  <div class="my_element">
    Some <strong>text</strong>
  </div>
`);
```

`toHaveProperty(property, value[, options])`

Expects the received `Target` to have its given property value match the given `value`. If no value is given: the matcher will instead check that the given property exists on the target.

- Parameters
 - `property`: `string`
 - `value`: `any`
 - `options`: `{ message?: string }`
- Examples

```
expect("button").toHaveProperty("tabIndex", 0);
expect("input").toHaveProperty("ontouchstart");
expect("script").toHaveProperty("src", "./index.js");
```

`toHaveRect(rect[, options])`

Expects the `DOMRect` of the received `Target` to match the given `rect` object. The `rect` object can either be:

If the resulting `rect` value is a node, then both nodes' rects will be compared.

- Parameters
 - `rect: Partial<DOMRect> | Target`
 - `options: { message?: string, trimPadding?: boolean }`
- Examples

```
expect("button").toHaveRect({ x: 20, width: 100, height: 50 });  
expect("button").toHaveRect(".container");
```

`toHaveStyle(style[, options])`

Expects the received `Target` to match the given style properties.

- Parameters
 - `style: string | Record<string, string | RegExp>`
 - `options: { message?: string }`
- Examples

```
expect("button").toHaveStyle({ color: "red" });  
expect("p").toHaveStyle("text-align: center");
```

`toHaveText(text[, options])`

Expects the `text` content of the received `Target` to either:

- be strictly equal to a given string;
- match a given regular expression.

Note: `innerHTML` is used to retrieve the text content to take CSS visibility into account. This also means that text values from child elements will be joined using a line-break as separator.

- Parameters
 - `text: string | RegExp`

```
expect("p").toHaveText("lorem ipsum dolor sit amet");  
expect("header h1").toHaveText(/odoo/i);
```

toHaveValue(value[, options])

Expects the value of the received `Target` to either:

- be strictly equal to a given string or number;
- match a given regular expression;
- contain file objects matching the given `files` list.

- Parameters

- `value`: any
- `options`: { `message?: string` }

- Examples

```
expect("input[type=email]").toHaveValue("john@doe.com");  
expect("input[type=file]").toHaveValue(new File(["foo"], "foo.txt"));  
expect("select[multiple]").toHaveValue(["foo", "bar"]);
```

Static methods

The `expect` helper function also contains static methods that can be used to run through a detached testing flow that isn't bound to one specific value at a certain moment.

These methods are mainly used to register steps or errors in the scope of the current test, and to evaluate them later on.

expect.assertions(expected)

Arguments:

expected (`number()`) –

Expects the current test to have the `expected` amount of assertions. This number cannot be less than 1.

Note

expect.errors (*expected*)

Arguments:

expected (`number()`) –

Expects the current test to have the `expected` amount of errors.

This also means that from the moment this function is called, the test will accept that amount of errors before being considered as failed.

expect.step (*value*)

Arguments:

value (`unknown()`) –

Registers a step for the current test, that can be consumed by `expect.verifySteps()`. Unconsumed steps will fail the test.

expect.verifyErrors (*errors*[, *options*])

Arguments:

errors (`unknown[]()`) –

options ({ `message?: string` }()) –

Returns:

`boolean`

Expects the received matchers to match the errors thrown since the start of the test or the last call to `expect.verifyErrors()`. Calling this matcher will reset the list of current errors.

```
expect.verifyErrors([/RPCError/, /Invalid domain AST/]);
```

expect.verifySteps (*steps*[, *options*])

Arguments:

steps (`unknown[]()`) –

options ({ `ignoreOrder?: boolean`, `message?: string`, `partial?: boolean` }()) –

Returns:

`boolean`

```
expect.step("web_read_group");
expect.step([1, 2]);
expect.verifySteps(["web_read_group", [1, 2]]);
```

expect.waitForErrors (errors[, options])

Arguments:

errors (unknown[]()) –

options ({ message?: string }()) –

Returns:

Promise<boolean>

Same as [expect.verifyErrors\(\)](#) , but will not immediately fail if errors are not caught yet, and will instead wait for a certain timeout (default: 2000ms) to allow errors to be caught later.

Checks are performed initially, at the end of the timeout, and each time an error is detected.

```
fetch("invalid/url");
await expect.waitForErrors([/RPCError/]);
```

expect.waitForSteps (steps[, options])

Arguments:

steps (unknown[]()) –

options ({ ignoreOrder?: boolean, message?: string, partial?: boolean }()) –

Returns:

Promise<boolean>

Same as [expect.verifySteps\(\)](#) , but will not immediately fail if steps have not been registered yet, and will instead wait for a certain timeout (default: 2000ms) to allow steps to be registered later.

Checks are performed initially, at the end of the timeout, and each time a step is registered.

```
// ... step on each 'web_read_group' call
fetch("../call_kw/web_read_group");
```

Custom DOM selectors

Here's a brief section on DOM selectors in Hoot, as they support additional pseudo-classes that can be used to target elements based on non-standard features, such as their text content or their global position in the document.

- **:contains(text)**
matches nodes whose text content matches the given `text`
 - given `text` supports regular expression syntax (e.g. `:contains(/^foo.+/)`) and is case-insensitive (unless using the `i` flag at the end of the regex)
- **:displayed**
matches nodes that are *"displayed"* (see `isDisplayed`)
- **:empty**
matches nodes that have an empty content (value or text content)
- **:eq(n)**
returns the *n*th node based on its global position (0-based index);
- **:first**
returns the first node matching the selector (in the whole document)
- **:focusable**
matches nodes that can be *"focused"* (see `isFocusable`)
- **:hidden**
matches nodes that are *not "visible"* (see `isVisible`)
- **:iframe**
matches nodes that are `<iframe>` elements, and returns their `body` if it is ready
- **:last**
returns the last node matching the selector (in the whole document)
- **:selected**
matches nodes that are selected (e.g. `<option>` elements)
- **:shadow**
matches nodes that have shadow roots, and returns their shadow root

matches nodes whose value matches the given `text`

- given `text` supports regular expression syntax (e.g. `:value(/^foo.+/)`) and is case-insensitive (unless using the `i` flag at the end of the regex)

- **`:visible`**

matches nodes that are “*visible*” (see `isVisible`)

Query & node properties helpers

Hoot provides helpers to query nodes and some of their properties in a streamlined and elegant way. This can mainly be done through the use of `queryX` helpers:

`queryAll(target[, options])`

Returns a list of nodes matching the given `Target`. This function can either be used as a **template literal tag** (only supports string selector without options) or invoked the usual way.

The target can be:

- a `Node` (or an iterable of nodes), or `Window` object;
- a `Document` object (which will be converted to its body);
- a string representing a **custom selector** (which will be queried from the `root` option).

An `options` object can be specified to filter **[1]** the results:

- `count`: the exact number of nodes to match (throws an error if the number of nodes doesn't match);
- `displayed`: whether the nodes must be “displayed” (see `isDisplayed`);
- `focusable`: whether the nodes must be “focusable” (see `isFocusable`);
- `root`: the root node to query the selector in (defaults to the current fixture);
- `visible`: whether the nodes must be “visible” (see `isVisible`). * This option implies `displayed`

[1] these filters (except for `count` and `root`) achieve the same result as using their homonym pseudo-classes on the final group of the given selector string, e.g.:

```
// These 2 will return the same result
queryAll`ul > li:visible`;
queryAll("ul > li", { visible: true });
```

`queryAllAttributes(target, attribute[, options])`

Performs a `queryAll()` on the given `target` and returns a list of attribute values.

Returns:

`string[]` list of attribute values

`queryAllProperties(target, property[, options])`

Performs a `queryAll()` on the given `target` and returns a list of property values.

Returns:

`unknown[]` list of property values

`queryAllTexts(target[, options])`

Performs a `queryAll()` on the given `target` and returns a list of text contents.

Returns:

`string[]` list of text contents

`queryAllValues(target[, options])`

Performs a `queryAll()` on the given `target` and returns a list of values.

Returns:

`string[]` a list of values

`queryAttribute(target, attribute[, options])`

Performs a `queryOne()` with the given arguments and returns the value of the given `attribute` of the matching node.

Returns:

`string` the attribute value

`queryFirst(target[, options])`

Performs a `queryAll()` with the given arguments and returns the first result or `null`.

Returns:

Performs a `queryAll()` with the given arguments, along with a forced `count: 1` option to ensure only one node matches the given `Target` .

The returned value is a single node instead of a list of nodes.

Returns:

Node a single node

`queryText(target[, options])`

Performs a `queryOne()` with the given arguments and returns the *text* of the matching node.

Returns:

string text of the matching node

`queryValue(target[, options])`

Performs a `queryOne()` with the given arguments and returns the *value* of the matching node.

Returns:

string value of the matching node

All of the above helpers are synchronous, meaning that they will attempt to query nodes instantly. Although some use cases require the element to be awaited for an arbitrary amount of time, unknown in advance due to UI fetching and rendering complexity.

Hoot provides 2 methods to wait for an element to appear / disappear within a certain time frame (by default: 200 milliseconds) for such cases:

`waitFor(target[, options])`

Combination of `queryAll()` and `waitUntil()` : waits for a given target to match elements in the DOM and returns the first matching node when it appears (or immediately if it is already present).

Returns:

Promise<Node> containing the first matching node

Returns:

Promise<number> containing the number of matching nodes

DOM: interaction helpers

Along with querying elements, it is often required to interact with them. As such, Hoot provides helpers to simulate various user interactions on elements.

These can be split into 2 types based on their parameters: **pointer-based** interaction helpers, and the **other** ones.

Pointer interaction helpers:

Pointer interaction helpers (such as `click()` or `drag()`) will simulate actual pointer movements and events on the given target, and on any previous element the pointer was *supposed* to have been.

`check(target[, options])`

Ensures that the given Target is checked.

If it is not checked, a `click()` is simulated on the input. If the input is still not checked after the click, an error is thrown.

Returns:

Promise<Event[]>

```
check("input[type=checkbox]"); // Checks the first <input> checkbox element
```

`click(target[, options])`

Performs a click sequence on the given Target .

The event sequence is as follows:

- pointerdown
- [desktop] mousedown
- [touch] touchstart
- [target is not active element] blur

- `[touch] touchend`
- `click`
- `dblclick` if click is not prevented & current click count is even

Returns:

Promise<Event[]>

```
click("button"); // Clicks on the first <button> element
```

`dblclick(target[, options])`

Performs two `click()` sequences on the given Target .

Returns:

Promise<Event[]>

```
dblclick("button"); // Double-clicks on the first <button> element
```

`drag(target[, options])`

Starts a drag sequence on the given Target .

Returns a set of helper functions to direct the sequence:

- `moveTo` : moves the pointer to the given target;
- `drop` : drops the dragged element on the given target (if any);
- `cancel` : cancels the drag sequence.

Returns:

Promise<DragHelpers>

```
drag(".card:first").drop(".card:last"); // Drags the first card onto the last one
```

```
drag(".card:first").moveTo(".card:last").drop(); // Same as above
```

```
const { cancel, moveTo } = await drag(".card:first"); // Starts the drag sequence
```


`hover(target[, options])`

Performs a hover sequence on the given Target .

The event sequence is as follows:

- `pointerover`
- `[desktop] mouseover`
- `pointerenter`
- `[desktop] mouseenter`
- `pointermove`
- `[desktop] mousemove`
- `[touch] touchmove`

Returns:

`Promise<Event[]>`

```
hover("button"); // Hovers the first <button> element
```

`pointerDown(target[, options])`

Performs a pointer down on the given Target .

The event sequence is as follows:

- `pointerdown`
- `[desktop] mousedown`
- `[touch] touchstart`
- `[target is not active element] blur`
- `[target is focusable] focus`

Returns:

`Promise<Event[]>`

```
pointerDown("button"); // Focuses to the first <button> element
```

`pointerUp(target[, options])`

- `pointerup`
- `[desktop] mouseup`
- `[touch] touchend`

Returns:

`Promise<Event[]>`

```
pointerUp("body"); // Triggers a pointer up on the <body> element
```

`scroll(target, position[, options])`

Performs a scroll event sequence on the given Target .

The event sequence is as follows:

- `[desktop] wheel`
- `scroll`

Returns:

`Promise<Event[]>`

```
scroll("body", { y: 0 }); // Scrolls to the top of <body>
```

`setInputRange(target, value[, options])`

Sets the given value to the current "input[type=range]" Target .

The event sequence is as follows:

- `pointerdown`
- `input`
- `change`
- `pointerup`

Returns:

`Promise<Event[]>`

`uncheck(target[, options])`

click, an error is thrown.

Returns:

Promise<Event[]>

```
uncheck("input[type=checkbox]"); // Unchecks the first <input> checkbox element
```

Other interaction helpers:

Other interaction helpers will not have a `target` parameter. It is not needed, since pressing keys on a keyboard (for example) is done on the current *active element*.

`clear([options])`

Clears the value of the current *active element*.

This is done using the following sequence:

- pressing "Control" & "A" to select the whole value;
- pressing "Backspace" to delete the value;
- (optional) triggering a "change" event by pressing "Enter" .

Returns:

Promise<Event[]>

```
clear(); // Clears the value of the current active element
```

`edit(value[, options])`

Combination of `clear()` and `fill()` :

- first, clears the input value (if any)
- then fills the input with the given value

Returns:

Promise<Event[]>

fill(value[, options])

Fills the current *active element* with the given `value`. This helper is intended for `<input>` and `<textarea>` elements, with the exception of "checkbox" and "radio" types, which should be selected using the `check` helper.

If the target is an editable input, its string `value` will be input one character at a time, each generating its corresponding keyboard event sequence. This behavior can be overridden by passing the `instantly` option, which will instead simulate a `control + v` keyboard sequence, resulting in the whole text being pasted.

Note that the given value is appended to the current value of the element.

If the active element is a `<input type="file"/>`, the `value` should be a `File` /list of `File` object(s).

Returns:

Promise<Event[]>

```
fill("Hello World"); // Types "Hello World" in the active element
fill("Hello World", { instantly: true }); // Pastes "Hello World" in the active e
fill(new File(["Hello World"], "hello.txt")); // Uploads a file named "hello.txt"
```

keyDown(keyStrokes[, options])

Performs a key down sequence on the current *active element*.

The event sequence is as follows:

- `keydown`

Additional actions will be performed depending on the key pressed:

- `Tab` : focus next (or previous with `shift`) focusable element;
- `c` : copy current selection to clipboard;
- `v` : paste current clipboard content to current element;
- `Enter` : submit the form if the target is a `<button type="button">` or a `<form>` element, or trigger a `change` event on the target if it is an `<input>` element;

Returns:

Promise<Event[]>

```
keyDown(" "); // Space key
```

keyUp(*keyStrokes* [, *options*])

Performs a key up sequence on the current *active element*.

The event sequence is as follows:

- keyup

Returns:

Promise<Event[]>

```
keyUp("Enter");
```

leave([*options*])

Performs a leave sequence on the current Window .

The event sequence is as follows:

- pointermove
- [desktop] mousemove
- [touch] touchmove
- pointerout
- [desktop] mouseout
- pointerleave
- [desktop] mouseleave

Returns:

Promise<Event[]>

```
leave("button"); // Moves out of <button>
```

The event sequence is as follows:

- `keydown`
- `keyup`

Returns:

`Promise<Event[]>`

```
pointerDown("button[type=submit]"); // Moves focus to <button>
keyDown("Enter"); // Submits the form

keyDown("Shift+Tab"); // Focuses previous focusable element

keyDown(["ctrl", "v"]); // Pastes current clipboard content
```

`resize([dimensions[, options]])`

Performs a resize event sequence on the current `Window`.

The event sequence is as follows:

- `resize`

The target will be resized to the given dimensions, enforced by `!important` style attributes.

Returns:

`Promise<Event[]>`

```
resize("body", { width: 1000, height: 500 }); // Resizes <body> to 1000x500
```

`select(value[, options])`

Performs a selection event sequence on the current active element. This helper is intended for `<select>` elements only.

The event sequence is as follows:

- `change`

Returns:

```
select("belgium"); // Selects the <option value="belgium"> element
```

setInputFiles(*files* [, *options*])

Gives the given `File` list to the current file input. This helper only works if a file input has been previously interacted with (by clicking on it).

Returns:

Promise<Event[]>

unload([*options*])

Triggers a "beforeunload" event on the current `Window`.

Returns:

Promise<Event[]>

Mocks

By default, a lot of low-level features are mocked by Hoot: `clipboard`, `fetch`, `localStorage`, etc. These mocks are intended to not produce any side-effect that would disturb the test runner or the context of other tests, while still providing the same interface to allow tests to rely on these features seamlessly.

There is also a need (most of the time) to force actions on these features or change their behavior for a test, so there exist helpers to interact with these mocked features. The following sections will list the main mocked features and the means to interact with them.

Time

Most asynchronous features are mocked: "timers" (`setTimeout`, `setInterval` and `requestAnimationFrame`), `Date` and `performance` all behave normally, but can be canceled or sped-up manually to considerably shorten the actual duration of tests. For example: all "timers" are canceled at the end of each test to avoid side-effects for the next one.

Important

There are 2 main timing behaviors that are *NOT* mocked:

- `Promise` objects and related API;

Related helpers

advanceFrame(*frameCount*)

Calls `advanceTime` with the duration it would take for a given `frameCount` amount of frames to have rendered in the UI (i.e. $(1000 / \text{current FPS}) \times \text{frame count}$).

advanceTime(*ms*)

Advances the current time by the given amount of milliseconds. This will affect all timeouts, intervals, animations and date objects.

It returns a promise resolved after all related callbacks have been executed.

Returns:

Promise<number> time consumed by timers (in ms)

animationFrame()

Returns a promise resolved after the next animation frame, typically allowing Owl components to render.

Returns:

Promise<void>

cancelAllTimers()

Cancels all current timeouts, intervals and animations.

delay()

Returns a promise resolved after a given amount of milliseconds (default to `0`).

```
await delay(1000); // waits for 1 second
```

Returns:

Promise<void>

microTick()

Returns a promise resolved after the next microtask tick.

mockDate(*date* [, *tz*])

Mocks the current date and time, and also the time zone if any.

Date can either be an object describing the date and time to mock, or a string in SQL or ISO format (time and millisecond values can be omitted). See [mockTimeZone\(\)](#) for the time zone params.

```
mockDate("2023-12-25T20:45:00"); // 2023-12-25 20:45:00 UTC
mockDate({ year: 2023, month: 12, day: 25, hour: 20, minute: 45 }); // same as above
mockDate("2019-02-11 09:30:00.001", +2);
```

mockTimeZone(*tz*)

Mocks the current time zone.

Time zone can either be a locale, a time zone or an offset.

Returns a function restoring the default zone.

```
mockTimeZone(+1); // UTC + 1
mockTimeZone("Europe/Brussels"); // UTC + 1 (or UTC + 2 in summer)
mockTimeZone("ja-JP"); // UTC + 9
```

runAllTimers([*preventTimers*])

Calculates the amount of time needed to run all current timeouts, intervals and animations, and then advances the current time by that amount.

Returns:

Promise<void>

setFrameRate(*frameRate*)

Sets the current frame rate (in fps) used by animation frames (default to 60fps).

tick()

Returns a promise resolved after the next task tick.

Returns:

Returns a promise fulfilled when the given `predicate` returns a truthy value, with the value of the promise being the return value of the `predicate` .

The `predicate` is run once initially and then each time the DOM is mutated (see `observe` for more information).

The promise automatically rejects after a given `timeout` (defaults to 5 seconds).

Returns:

Promise<unknown> promise with the return value of the predicate

Network

In general, we don't want to perform actual network calls in tests. To ensure this, all calls to `fetch` and `XMLHttpRequest` have been re-routed to a function given to `mockFetch()` .

Note

In Odoo, this is generally implicitly handled by a `MockServer` which is spawned by the mock environment, i.e. any time a component is rendered using the `mountWithCleanup` helper.

Related helpers

`mockFetch([fetchFn])`

Mocks the `fetch` function by replacing it with a given `fetchFn` .

The return value of `fetchFn` is used as the response of the mocked `fetch`, or wrapped in a `MockResponse` object if it does not meet the required format.

```
mockFetch((input, init) => {
  if (input === "../web_search_read") {
    return { records: [{ id: 3, name: "john" }] };
  }
  // ...
});
mockFetch((input, init) => {
  if (input === "/translations") {
    const translations = {
```

```
    }
  });
```

mockWebSocket ([onWebSocketConnected])

Activates mock WebSocket classe:

- websocket connections will be handled by `window.fetch` (see [mockFetch\(\)](#));
- the `onWebSocketConnected` callback will be called after a websocket has been created.

mockWorker ([onWorkerConnected])

Activates mock Worker and SharedWorker classes:

- actual code fetched by worker URLs will then be handled by `window.fetch` (see [mockFetch\(\)](#));
- the `onWorkerConnected` callback will be called after a worker has been created.

Notable global features

The following features may not have any specific mocked feature added, but they do work as expected without changing the actual properties they were meant to:

- [Document](#)

Both `title` and `cookie` can be set and read without changing the actual properties of the current document.

- [History](#)

The `history` API is mocked and bound to the `mockLocation` object to return the same values and provide consistency.

- [Location](#)

Hoot returns a `mockLocation` object to use instead of `window.location` , but this relies on the use of an indirection in the actual production code.

Important

This feature will only work if an indirection is set between production code and calls to `window.location` . In Odoo, it works because the `@web/core/browser` module

- **Navigator** [↗](#)

Most used navigator features, such as the `clipboard` API and `userAgent`, have been mocked to hijack their actual behaviors. Its `permissions` object has been bound to a global mock of the permissions API.

- **Notification** [↗](#)

Notifications have been mocked, with the "notification" permissions bound to the global mocked permissions API.

- **Permissions** [↗](#)

Permissions can enable or disable other APIs by being given the "granted" or "denied" statuses. This can be done through the `mockPermission` helper.

- **Storage** [↗](#)

`localStorage` and `sessionStorage` both point to "virtual" storages.

- **Touch** [↗](#)

Touch features can be force-activated or deactivated globally for a given test/suite using the `mockTouch()` helper. It will mock both the presence of touch handlers like `ontouchstart` on window, as well as the "pointer" media being set to `fine` or `coarse`.

Related helpers

`mockPermission(name[, value])`

Sets the given value for the given permission. This allows to enable or prevent certain APIs (see the [Permissions API](#) [↗](#)).

```
// Prevents the whole notification API from working
mockPermission("notifications", "denied");
```

`mockTouch(setTouch)`

Toggles touch features on or off in the current `Window`.



Get Help

Contact Support

Ask the Odoo Community

